

Tópico 05 - Parte 2: Mural de Contatos Persistente

UC2 – Banco de Dados Relacional

Prof. Jeronimo Silva

Roteiro da Aula

- **Bloco 1:** Checklist de Conceitos da Aula
- **Bloco 2:** Memória Volátil (UC1) vs Persistência SQLite (UC2)
- **Bloco 3:** Arquitetura em 4 Camadas (`conexao` ↔ `main` ↔ `preload` ↔ `renderer`)
- **Bloco 4:** Canais IPC & Sanitização com Regex
- **Bloco 5:** Roteiro da Prática Guiada (Code-Along)
- **Bloco 6:** Desafio Prático & Encerramento



Checklist de Conceitos da Aula

- **IPC (`invoke` / `handle`)**: Comunicação assíncrona bidirecional segura entre processos.
- **`better-sqlite3`**: Execução síncrona e performática de queries no Node.js.
- **`AUTOINCREMENT`**: Geração automática de ID numérico sequencial único no banco.
- **`UNIQUE`**: Restrição SQL que previne cadastros duplicados (ex: e-mail).
- **Sanitização com Regex**: Remoção de caracteres não numéricos.
- **`confirm()`**: Modal nativo do navegador para validação prévia de exclusão.

📞 Mural no DOM (UC1) vs SQLite (UC2)

✗ Limitação do DOM (UC1):

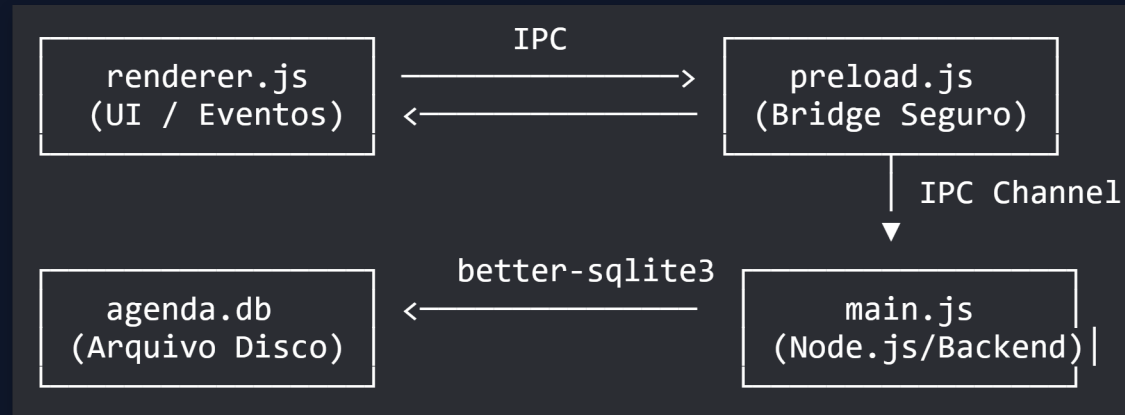
- Contatos armazenados no estado da página (`innerHTML += ...`).
- Ao recarregar (F5) ou fechar a janela, tudo é perdido (volatilidade da RAM).

✅ Solução Persistente no Electron (UC2):

- Dados gravados em arquivo físico no disco (`database/agenda.db`).
- Queries SQL garantem ordenação, busca eficiente e integridade relacional.



Arquitetura da Aplicação em 4 Camadas



- **Isolamento de Segurança:** O Frontend (**renderer.js**) não acessa o arquivo **.db** diretamente.



Modelagem da Tabela contatos

Nossa tabela no banco de dados possuirá as seguintes colunas:

- **id** : Chave primária de tipo número inteiro com auto-incremento.
- **nome** : Texto obrigatório.
- **foto_url** : Texto para armazenar a URL da imagem de perfil.
- **telefone** : Texto obrigatório sanitizado.
- **email** : Texto único e obrigatório.



Mapeamento dos Canais IPC

Canal	Método	Descrição
<code>get-contatos</code>	Consulta (Leitura)	Retorna a lista de todos os contatos por ordem de nome.
<code>add-contato</code>	Inserção (Escrita)	Grava um novo contato no banco de dados.
<code>delete-contato</code>	Exclusão (Remoção)	Remove um contato fisicamente do banco pelo seu ID.

Sanitização com Regex & Confirmação

1. Limpeza de Telefone (Regex):

- A expressão `replace(/\D/g, '')` remove tudo que não for número do campo digitado.

2. Confirmação Visual Pré-Deleção:

- Uso do método nativo `confirm(...)` para solicitar confirmação do usuário antes de disparar a rota de exclusão no IPC.

Roteiro da Prática Guiada (Code-Along)

1. Passo 1 (`conexao.js`): Configurar a conexão do `better-sqlite3` e criar a tabela `contatos` .
2. Passo 2 (`main.js`): Registrar os 3 handlers IPC (`get-contatos` , `add-contato` , `delete-contato`).
3. Passo 3 (`preload.js`): Expor a ponte de contexto segura em `window.api` .
4. Passo 4 (`renderer.js`): Conectar os formulários do DOM e implementar a renderização e exclusão.

Desafio Prático & Próximos Passos

Desafio da Aula (Mural Avançado):

1. **Filtro em Tempo Real (LIKE):** Crie a busca em tempo real conforme o usuário digita.
2. **Validação de E-mail:** Trate erros de e-mail duplicado exibindo aviso amigável.

Próxima Aula:

- **Tópico 06:** Desafio Kanban Persistente (Arrastar e soltar com status no SQLite).